

AI-Powered Enterprise E-Commerce Platform

Deployment Architecture Specification

Document Information

Item	Description
Project Name	AI-Powered Enterprise E-Commerce Platform
Document Type	Deployment Architecture
Version	1.0
Prepared By	Project Team
Status	Draft

Revision History

Version	Date	Description
1.0	DD/MM/YYYY	Initial Version

Table of Contents

- 1. Introduction
- 2. Purpose
- 3. Deployment Objectives
- 4. Deployment Environments
- 5. High-Level Deployment Architecture
- 6. Container Architecture
- 7. Infrastructure Components
- 8. Network Architecture
- 9. Scalability Strategy
- 10. Availability Strategy
- 11. Monitoring and Logging
- 12. Backup and Recovery
- 13. Security Considerations
- 14. CI/CD Integration
- 15. Cloud Migration Strategy
- 16. Future Enhancements

17. Conclusion

18. References

1. Introduction

This document defines the deployment strategy for the AI-Powered Enterprise E-Commerce Platform.

The platform consists of multiple services:

- React Frontend
- Spring Boot Backend Services
- FastAPI AI Services
- PostgreSQL
- Redis
- RabbitMQ

The deployment architecture aims to ensure:

- reliability,
- scalability,
- maintainability,
- and high availability.

2. Purpose

The purpose of this document is to:

- define deployment environments,
- describe infrastructure components,
- establish deployment standards,
- support future cloud deployment.

3. Deployment Objectives

DEP-01

Provide stable environments.

DEP-02

Support scalability.

DEP-03

Minimize downtime.

DEP-04

Simplify deployments.

DEP-05

Support future cloud migration.

4. Deployment Environments

The system will use multiple environments.

Development Environment

Purpose:

Developer testing.

Components:

- Local Docker Containers
 - Local PostgreSQL
 - Local Redis
-

Staging Environment

Purpose:

Pre-production validation.

Components:

- Docker Compose Deployment
 - Production-like configurations
-

Production Environment

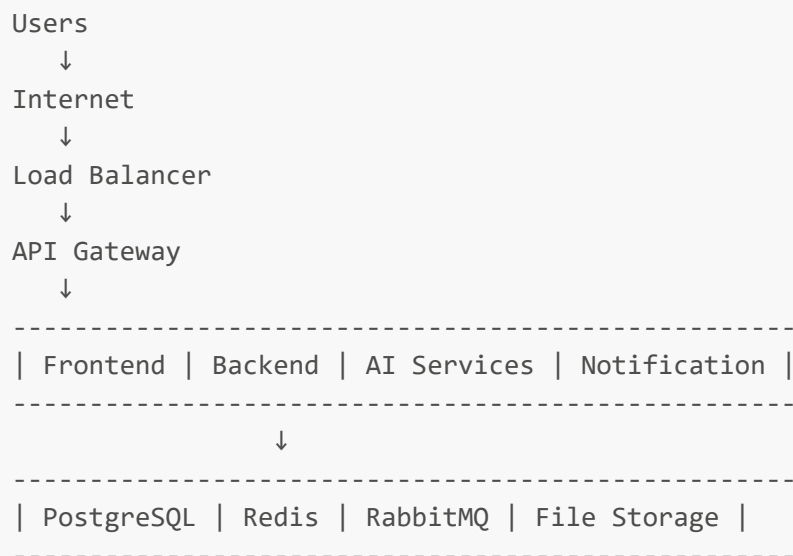
Purpose:

Real deployment.

Components:

- Containerized Services
- Monitoring Stack
- Backup Services

5. High-Level Deployment Architecture



6. Container Architecture

All major services shall run inside containers.

Frontend Container

Technology:

- React
- NGINX

Responsibilities:

- UI Rendering
- API Communication

Backend Container

Technology:

- Spring Boot

Responsibilities:

- Business Logic
 - Authentication
 - Product Management
 - Orders
-

AI Services Container

Technology:

- FastAPI

Responsibilities:

- Recommendations
 - Fraud Detection
 - Forecasting
 - Customer Segmentation
-

Database Container

Technology:

- PostgreSQL

Responsibilities:

- Persistent Data Storage
-

Cache Container

Technology:

- Redis

Responsibilities:

- Session Storage
 - Recommendation Cache
-

Messaging Container

Technology:

- RabbitMQ

Responsibilities:

- Asynchronous Communication

7. Docker Deployment Architecture

Development Deployment

```
docker-compose up
```

Services:

```
frontend  
backend  
ai-services  
postgres  
redis  
rabbitmq
```

Example Container Structure

```
frontend-container  
backend-container  
recommendation-container  
fraud-container  
forecast-container  
postgres-container  
redis-container  
rabbitmq-container
```

8. Infrastructure Components

Load Balancer

Purpose:

Distribute requests.

Possible Technologies:

- NGINX
 - AWS ALB
-

API Gateway

Responsibilities:

- Authentication
 - Routing
 - Logging
 - Rate Limiting
-

Database Server

Responsibilities:

- Data Persistence
-

Monitoring Server

Responsibilities:

- Metrics Collection
 - Visualization
-

9. Network Architecture

External Network

Accessible Components:

- Frontend
 - API Gateway
-

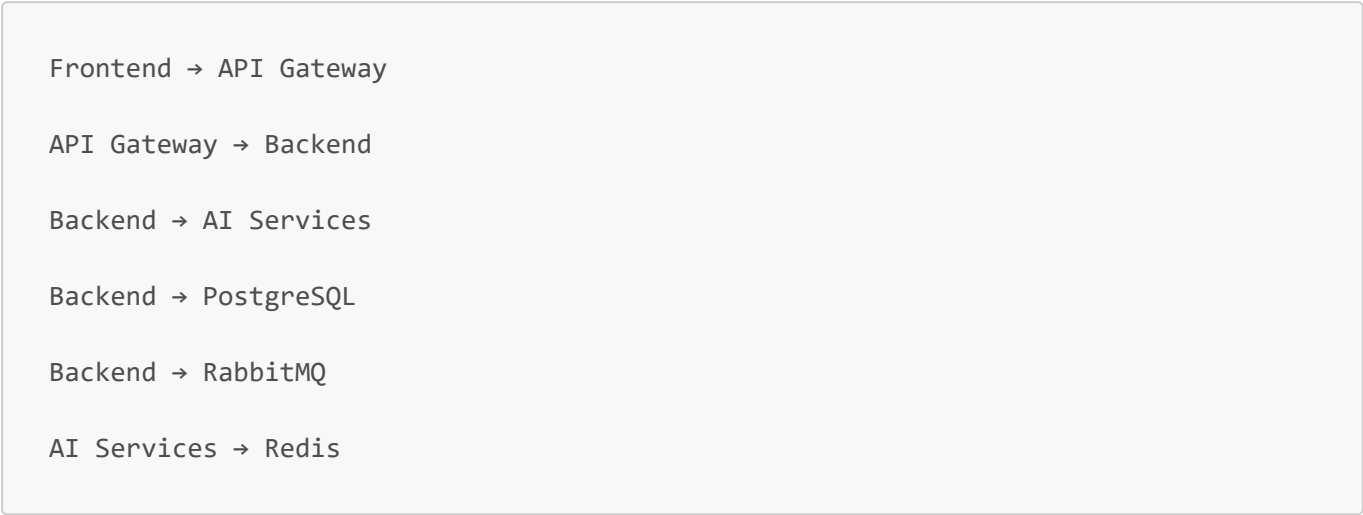
Internal Network

Accessible Components:

- Backend Services

- AI Services
- Databases

Communication Rules



10. Deployment Diagram



11. Availability Strategy

Target:

99.9% Availability

Strategies

- Health Checks
 - Container Restart Policies
 - Backups
 - Monitoring
 - Redundant Services
-

12. Scalability Strategy

Horizontal Scaling

Possible Services:

- Backend
 - Recommendation Services
 - Frontend
-

Vertical Scaling

Increase:

- CPU
 - Memory
 - Storage
-

Database Scaling

Future:

- Read Replicas
 - Partitioning
 - Sharding
-

13. Monitoring Architecture

Metrics Collection

Technology:

Prometheus

Visualization

Technology:

Grafana

Logs

Technology:

ELK Stack

Metrics to Monitor

- CPU Usage
 - Memory Usage
 - API Response Time
 - Error Rates
 - Database Connections
 - AI Model Latency
-

14. Backup Architecture

Database Backups

Frequency:

Daily

Incremental Backups

Every few hours.

Backup Storage

Separate secure location.

Recovery Objectives

RTO:

< 4 Hours

RPO:

< 30 Minutes

15. Security Considerations

Network Security

- HTTPS
 - Internal Network Isolation
-

Container Security

- Non-root Containers
 - Minimal Base Images
-

Secrets Management

Store:

- Database Passwords
 - API Keys
 - JWT Secrets
-

Access Controls

Role-based administrative access.

16. CI/CD Integration

Deployment pipeline:

```
graph TD
    A[Developer Push] --> B[GitHub Actions]
    B --> C[Build]
    C --> D[Tests]
    D --> E[Docker Build]
    E --> F[Deployment]
```

17. Cloud Deployment Strategy

Phase 1

Local Docker Deployment.

Phase 2

Cloud VM Deployment.

Possible Platforms:

- AWS EC2
 - Azure VM
-

Phase 3

Container Orchestration.

Technology:

Kubernetes.

Phase 4

Enterprise Deployment.

Features:

- Auto Scaling
 - Managed Databases
 - Multi-AZ Deployment
-

18. Deployment Risks

Risk	Mitigation
Container Failure	Restart Policies
Database Failure	Backups
Network Failure	Monitoring
Resource Exhaustion	Auto Scaling
Misconfiguration	Infrastructure Documentation

19. Infrastructure Requirements

Minimum Development Environment

CPU:

4 Cores

Memory:

16 GB Recommended

Storage:

50 GB+

Suggested Production Environment

CPU:

8+ Cores

Memory:

32 GB+

Storage:

200 GB+

20. Future Enhancements

Possible future improvements:

- Kubernetes Deployment
 - Service Mesh (Istio)
 - Multi-region Deployment
 - CDN Integration
 - Infrastructure as Code
 - Blue-Green Deployment
 - Canary Releases
-

21. Recommended Directory Structure

```
deployment/
├── docker/
│   ├── docker-compose.yml
│   ├── Dockerfile.frontend
│   ├── Dockerfile.backend
│   └── Dockerfile.ai
├── kubernetes/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── ingress.yaml
└── monitoring/
    ├── prometheus.yml
    └── grafana/
```

22. Conclusion

The proposed deployment architecture provides:

- reliability,
- scalability,
- maintainability,
- security,
- and future cloud readiness.

The architecture enables gradual evolution from local deployment to enterprise cloud-native deployment while maintaining flexibility and operational efficiency.

References

- [1] Docker Documentation.
- [2] Kubernetes Documentation.
- [3] NGINX Documentation.
- [4] Newman, S. (2021). Building Microservices.
- [5] Bass, L., Clements, P., & Kazman, R. Software Architecture in Practice.
- [6] AWS Well-Architected Framework.
- [7] Google Site Reliability Engineering (SRE) Book.